15-10-2026

CO1 -ASSSESMENT TOOL 3

CODING CHALLENGE

COURSE NAME: NATURAL LANGUAGE PROCESSING

COURSECODE: DSA0308

NAME: R SAI HASNIKA **REG NO:192424178**

Section 1: User Registration Validation Using Regular Expressions

Aim

To develop a Python program that validates an email address, password, and mobile number using regular expressions.

Algorithm

1. Import the re module.
2. Read the email address, password, and mobile number from the user.
3. Use regular expressions to validate each input according to the given rules.
4. Display whether the email is valid, the password is strong, and the mobile number is valid.

PSEUDO CODE

BEGIN

IMPORT regular expression module

INPUT email address

INPUT password

INPUT mobile number

```
// Validate Email
SET email_pattern to a pattern that:
   Starts with an alphabet
   Allows letters, digits, dot and underscore before @
   Allows only alphabets in the domain
   Ends with .com, .org, .edu, .net or .in
IF email matches email_pattern THEN
   DISPLAY "Valid Email"
ELSE
   DISPLAY "Invalid Email"
END IF
// Validate Password
SET password_pattern to a pattern that checks:
   Minimum 8 characters
   At least one uppercase letter
   At least one lowercase letter
   At least one digit
   At least one special character from @, #, $, %, &, !
IF password matches password_pattern THEN
   DISPLAY "Strong Password"
ELSE
   DISPLAY "Weak Password"
END IF
// Validate Mobile Number
SET mobile_pattern to a pattern that checks:
```

Exactly 10 digits

First digit is between 6 and 9

IF mobile number matches mobile_pattern THEN

DISPLAY "Valid Mobile Number"

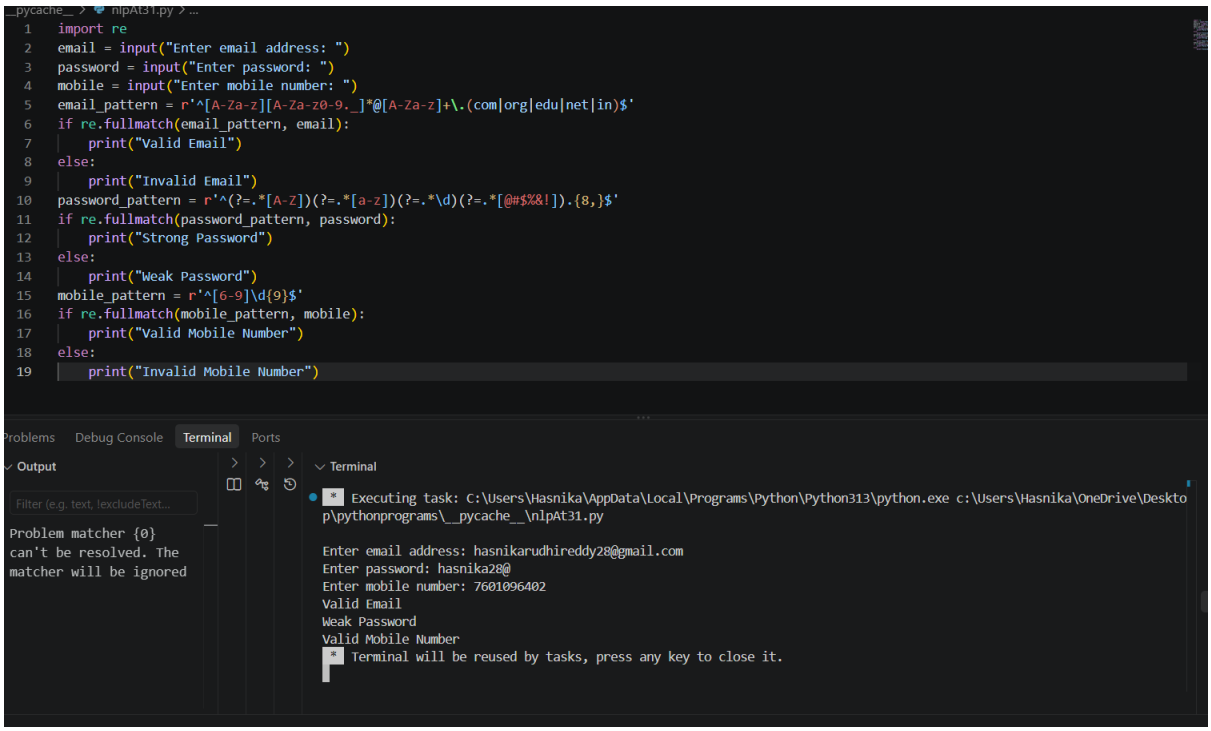
ELSE

DISPLAY "Invalid Mobile Number"

END IF

END

OUTPUT:



RESULT :

Section 2: Design a Finite State Automata (FSA) Simulator

Aim

To develop a Python program that simulates a Deterministic Finite Automaton (DFA) by accepting its states, input alphabet, transition table, initial state, and final states, then determines whether input strings are accepted or rejected.

Algorithm

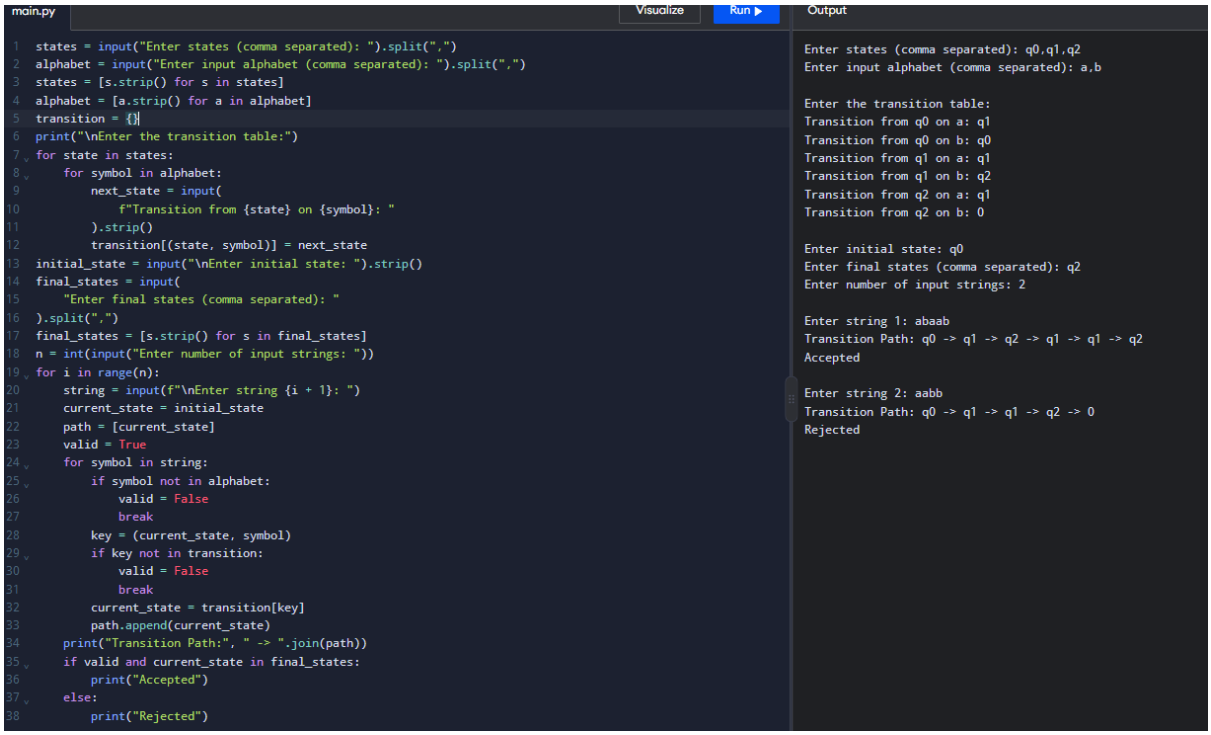
1. Read the set of states and input alphabet.
2. Create a transition table for every state and input symbol.
3. Read the initial state and final states.
4. Accept multiple input strings from the user.
5. Start from the initial state and process each character using the transition table.
6. Display the transition path.
7. If the final state belongs to the set of final states, display Accepted; otherwise, display Rejected.

PSEUDO CODE:

```
BEGIN
  INPUT states
  INPUT input alphabet
  FOR each state in states
    FOR each symbol in input alphabet
      INPUT next state
      STORE transition in transition table
    END FOR
  END FOR
```

```
    INPUT initial state
    INPUT final states
    INPUT number of input strings
    FOR each input string
        SET current state = initial state
        SET transition path = initial state
        SET valid = TRUE
        FOR each symbol in input string
            IF symbol is not in input alphabet THEN
                SET valid = FALSE
                EXIT loop
            END IF
            FIND next state from transition table
            SET current state = next state
            ADD current state to transition path
        END FOR
        DISPLAY transition path
        IF valid is TRUE AND current state is a final state THEN
            DISPLAY "Accepted"
        ELSE
            DISPLAY "Rejected"
        END IF
    END FOR
END
```

CODE AND OUPUT:



Section 3: Smart Pattern Matching Engine Using Regular Expressions

Aim

To develop a Python-based text search engine using regular expressions to search for words, prefixes, suffixes, dates, phone numbers, hashtags, and mentions in a given text.

Algorithm

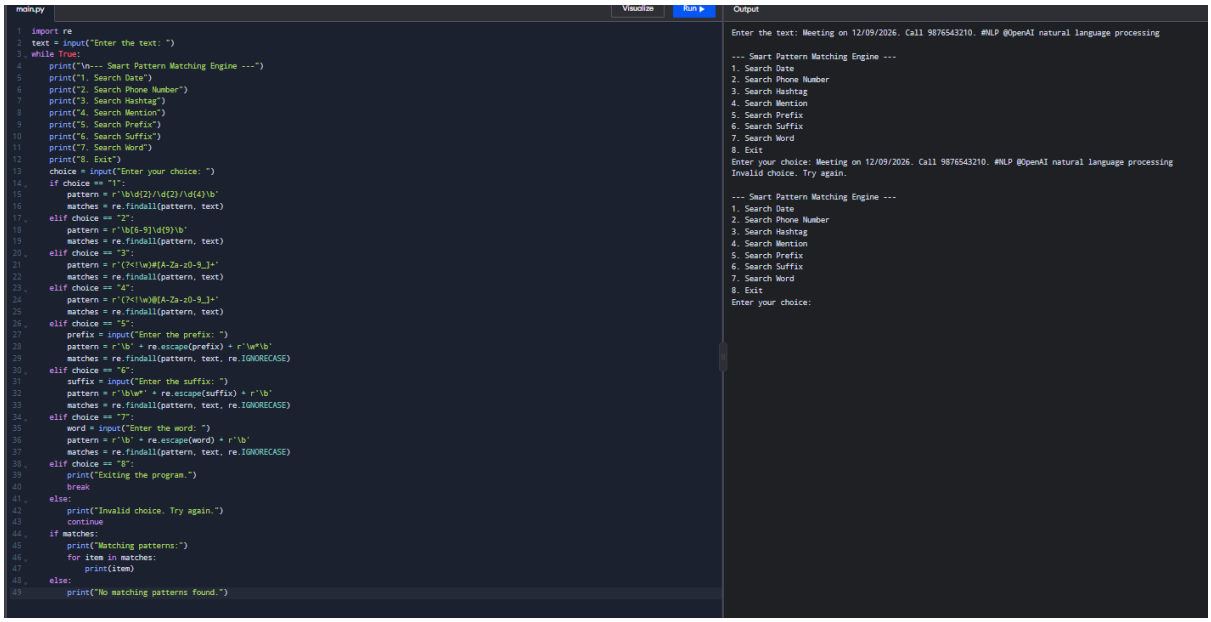
1. Import the re module.
2. Read a text paragraph from the user.
3. Display a menu containing different search options.
4. Ask the user to select a search operation.
5. Apply the appropriate regular expression to find matching patterns.
6. Display all matching results or a message if no matches are found.
7. Repeat the process until the user chooses to exit.

PSEUDO CODE:

```
BEGIN
    IMPORT regular expression module
    INPUT text
    REPEAT
        DISPLAY "1. Search Date"
        DISPLAY "2. Search Phone Number"
        DISPLAY "3. Search Hashtag"
        DISPLAY "4. Search Mention"
        DISPLAY "5. Search Prefix"
        DISPLAY "6. Search Suffix"
        DISPLAY "7. Search Word"
        DISPLAY "8. Exit"
        INPUT choice
        IF choice = 1 THEN
            SET pattern to match dates in DD/MM/YYYY format
        ELSE IF choice = 2 THEN
            SET pattern to match 10-digit mobile numbers
            starting with a digit from 6 to 9
        ELSE IF choice = 3 THEN
            SET pattern to match hashtags
        ELSE IF choice = 4 THEN
            SET pattern to match @username mentions
        ELSE IF choice = 5 THEN
            INPUT prefix
            SET pattern to match words starting with prefix
```

```
        ELSE IF choice = 6 THEN
            INPUT suffix
            SET pattern to match words ending with suffix
        ELSE IF choice = 7 THEN
            INPUT word
            SET pattern to match the complete word
        ELSE IF choice = 8 THEN
            DISPLAY "Exiting the program"
            EXIT loop
        ELSE
            DISPLAY "Invalid choice"
            CONTINUE loop
        END IF
        FIND all matches in the text using the pattern
        IF matches are found THEN
            DISPLAY all matching patterns
        ELSE
            DISPLAY "No matching patterns found"
        END IF
    UNTIL choice = 8
END
```

CODE AND OUTPUT:



```
import re
text = input("Enter the text: ")
while True:
    print("\n--- Smart Pattern Matching Engine ---")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Search Word")
    print("8. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        pattern = r'\b\d{2}/\d{2}/\d{4}\b'
        matches = re.findall(pattern, text)
    elif choice == "2":
        pattern = r'\b[6-9]\d{9}\b'
        matches = re.findall(pattern, text)
    elif choice == "3":
        pattern = r'(?<!\w)#[A-Za-z0-9_]+'
        matches = re.findall(pattern, text)
    elif choice == "4":
        pattern = r'(?<!\w)@[A-Za-z0-9_]+'
        matches = re.findall(pattern, text)
    elif choice == "5":
        prefix = input("Enter the prefix: ")
        pattern = r'\b' + re.escape(prefix) + r'\w*\b'
        matches = re.findall(pattern, text, re.IGNORECASE)
    elif choice == "6":
        suffix = input("Enter the suffix: ")
        pattern = r'\b\w*' + re.escape(suffix) + r'\b'
        matches = re.findall(pattern, text, re.IGNORECASE)
    elif choice == "7":
        word = input("Enter the word: ")
        pattern = r'\b' + re.escape(word) + r'\b'
        matches = re.findall(pattern, text, re.IGNORECASE)
    elif choice == "8":
        print("Exiting the program.")
        break
    else:
        print("Invalid choice. Try again.")
        continue
    if matches:
        print("Matching patterns:")
        for item in matches:
            print(item)
    else:
        print("No matching patterns found.")
```

```
Enter the text: Meeting on 12/09/2026. Call 9876543210. #NLP @OpenAI natural language processing

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: Meeting on 12/09/2026. Call 9876543210. #NLP @OpenAI natural language processing
Invalid choice. Try again.

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice:
```